

Allow pack expansion in lambda init-capture

Contents

- [Motivation](#)
- [Init-capture restriction history](#)
- [Proposal](#)
- [Example](#)
- [Revision History](#)
- [Acknowledgements](#)
- [References](#)

Motivation

With the introduction of generalized lambda capture [1], lambda captures can be nearly arbitrarily complex and solve nearly all problems. However, there is still an awkward hole in the capabilities of lambda capture when it comes to parameter packs: you can only capture packs by copy, by reference, or by... `std::tuple`?

Consider the simple example of trying to wrap a function and its arguments into a callable to be accessed later. If we copy everything, the implementation is both easy to write and read:

```
template<class F, class... Args>
auto delay_invoke(F f, Args... args) {
    // the capture here can also be just [=]
    return [f, args...]() -> decltype(auto) {
        return std::invoke(f, args...);
    };
}
```

But if we try to be more efficient about the implementation and try to `move` all the arguments into the lambda? It seems like you should be able to use an *init-capture* and write:

```
template<class F, class... Args>
auto delay_invoke(F f, Args... args) {
    return [f=std::move(f), ...args=std::move(args)]() -> decltype(auto) {
        return std::invoke(f, args...);
    };
}
```

But this runs afoul of very explicit wording from [\[expr.prim.lambda.capture\]/17](#), emphasis mine:

A simple-capture followed by an ellipsis is a pack expansion. An init-capture followed by an ellipsis is ill-formed.

As a result of this restriction, our only option is to put all the `args...` into a `std::tuple`. But once we do that, we don't have access to the arguments as a parameter pack, so we need to pull them back out of the tuple in the body, using something like `std::apply()`:

```
template<class F, class... Args>
auto delay_invoke(F f, Args... args) {
    return [f=std::move(f), tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) {
        return std::apply(f, tup);
    };
}
```

```
};
}
```

Which gets even worse if what we wanted to do with that captured parameter pack was invoke a *named* function rather than a captured object. At that point, all semblance of comprehension goes out the window:

By copy	By move
<pre>template <class... Args> auto delay_invoke_foo(Args... args) { return [args...]() -> decltype(auto) { return foo(args...); }; }</pre>	<pre>template <class... Args> auto delay_invoke_foo(Args... args) { return [tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) { return std::apply([](auto const&... args) -> decltype(auto) { return foo(args...); }, tup); }; }</pre>

We can do better.

Init-capture restriction history

The explicit restriction on pack expansion in *init-capture* was in the initial wording paper for generalized lambda capture [2], due in part to the rules proposed in that paper as to how *init-capture* would work. The original wording read:

For every *init-capture* a non-static data member named by the *identifier* of the *init-capture* is declared in the closure type. This member is not a bit-field and not mutable. The type of that member corresponds to the type of a hypothetical variable declaration of the form "auto *init-capture* ";, except that the variable name (i.e., the *identifier* of the *init-capture*) is replaced by a unique identifier.

Which introduces a problem, as explained by Richard Smith in [3]:

One problem here is that an *init-capture* introduces a **named** member of the closure type. A class member name that names a pack would be a new notion, and would bring with it significant additional complications (such as the inability to determine syntactically whether a construct contains an unexpanded parameter pack).

[...]

Consider this:

```
template <typename T> void call_f(T t) {
    f(t.x...);
}
```

Right now, this is ill-formed (no diagnostic required) because "t.x" does not contain an unexpanded parameter pack. But if we allow class members to be pack expansions, this code could be valid -- we'd lose any syntactic mechanism to determine whether an expression contains an unexpanded pack. This is fatal to at least one implementation strategy for variadic templates. It also admits the possibility of pack expansions occurring outside templates, which current implementations are not well-suited to handle.

Since *init-captures* add named members to the closure type, allowing *init-captures* to be pack expansions risks introducing the same problem if those names are visible in **any** context outside the body of the lambda-expression itself.

However, this problem went away with the adoption of CWG 1760 [4], which changed the wording from *init-capture* introducing a *named* member (of unspecified access) to introducing an *unnamed* member. Once *init-capture* doesn't give us named members, the problem that was pointed out in [3] is no longer a problem. There would be no named pack member to give complications in parsing, so this proposal claims that this restriction is no longer necessary.

Proposal

The proposal is to simply remove the restriction on pack expansions in *init-capture*, which requires defining a new form of parameter pack in the language. Proposed wording is as follows.

In 6 [basic] paragraph 3:

An *entity* is a value, object, reference, function, enumerator, type, class member, bit-field, template, template specialization, namespace, or **parameter** pack.

In 8.4.5.2 [expr.prim.lambda.capture], change the grammar:

```
capture-list:
    capture---opt
    capture-list, capture---opt

capture:
    simple-capture---opt
---opt init-capture
```

In 8.4.5.2 [expr.prim.lambda.capture] paragraph 17:

A *simple-capture* followed by an ellipsis is a pack expansion (17.6.3 [temp.variadic]). ~~An *init-capture* followed by an ellipsis is ill-formed. An *init-capture* preceded by an ellipsis is a pack expansion that introduces an *init-capture pack* (17.6.3 [temp.variadic]) whose declarative region is the *lambda-expression's compound-statement*.~~ [Example:

```
template<class... Args>
void f(Args... args) {
    auto lm = [&, args...] { return g(args...); };
    lm();

    auto lm2 = [...xs=std::move(args)] { return g(xs...); };
    lm2().;
}
```

- end example]

In 8.4.6 [expr.prim.fold]:

A fold expression performs a fold of a **template parameter** pack (17.6.3 [temp.variadic]) over a binary operator.

Unary left folds and unary right folds are collectively called unary folds. In a unary fold, the cast-expression shall contain an unexpanded **parameter** pack (17.6.3 [temp.variadic]).

An expression of the form (e1 op1 ... op2 e2) where op1 and op2 are fold-operators is called a binary fold. In a binary fold, op1 and op2 shall be the same fold-operator, and either e1 shall contain an unexpanded **parameter** pack or e2 shall contain an unexpanded **parameter** pack, but not both. If e2 contains an unexpanded **parameter** pack, the expression is called a binary left fold. If e1 contains an unexpanded **parameter** pack, the expression is called a binary right fold.

In 8.5.2.3 [expr.sizeof] paragraph 5:

The identifier in a `sizeof...` expression shall name a **parameter** pack. The `sizeof...` operator yields the number of **arguments provided for elements in** the **parameter** pack **identifier** (17.6.3 [temp.variadic]).

In 11.3.5 [dcl.fct], paragraph 17:

A declarator-id or abstract-declarator containing an ellipsis shall only be used in a parameter-declaration. ~~Such a parameter-declaration is a parameter pack.~~ When it is part of a parameter-declaration-clause, the **parameter pack is** *parameter-declaration declares* a function parameter pack. ~~[Note: Otherwise, the parameter-declaration is part of a template-parameter-list and the parameter pack is parameter-declaration declares a template parameter pack; see [temp.param]. —end note]~~ A function parameter pack is a pack expansion.

In 11.3.6 [dcl.fct.default], paragraph 3:

A default argument shall be specified only in the parameter-declaration-clause of a function declaration or lambda-declarator or in a template-parameter; in the latter case, the initializer-clause shall be an assignment-expression. A default argument shall not be specified for a **template** parameter pack **or a function parameter pack**. If it is specified in a parameter-declaration-clause, it shall not occur within a declarator or abstract-declarator of a parameter-declaration.

In 17.1 [temp.param], paragraph 19:

If a *template-parameter* is a *type-parameter* with an ellipsis prior to its optional identifier or is a *parameter-declaration* that declares a **parameter** pack ([dcl.fct]), then the template-parameter is a template parameter pack. A template parameter pack that is a parameter-declaration whose type contains one or more unexpanded **parameter** packs is a pack expansion. Similarly, a template parameter pack that is a type-parameter with a template-parameter-list containing one or more unexpanded parameter packs is a pack expansion. A template parameter pack that is a pack expansion shall not expand a **template** parameter pack declared in the same template-parameter-list.

In 17.3.3 [temp.arg.template], paragraph 3:

If P contains a **template** parameter pack, then A also matches P if each of A's template parameters matches the corresponding template parameter in the template-head of P.

In 17.3.3 [temp.arg.template], paragraph 4.2:

Each function template has a single function parameter whose type is a specialization of X with template arguments corresponding to the template parameters from the respective function template where, for each template parameter PP in the template-head of the function template, a corresponding template argument AA is formed. If PP declares a **template** parameter pack, then AA is the pack expansion PP... ([temp.variadic]); otherwise, AA is the id-expression PP.

Add a new clause to 17.6.3 [temp.variadic] after paragraph 2:

An *init-capture pack* introduces an *init-capture* for each of the elements in the pack expansion of its *initializer*. [Example:

```
template <typename... Args>  
void foo(Args... args){  
    [...xs=args]{  
        bar(xs...); // xs is an init-capture pack  
    };  
}  
  
foo(); // OK: xs contains zero init-captures  
foo(1); // OK: xs contains one init-capture
```

- end example]

In 17.6.3 [temp.variadic], paragraph 3:

A **parameter** pack is **either** a template parameter pack, **or** a function parameter pack, or an *init-capture pack*. The number of elements of a template parameter pack or a function parameter pack is the number of arguments provided for the parameter pack identifier. The number of elements of an *init-capture pack* is the number of elements in the pack expansion of its initializer.

The section describing pack expansions in 17.6.3 [temp.variadic] paragraph 4 remains unchanged:

Pack expansions can occur in the following contexts:

- In a *capture-list* ([expr.prim.lambda]); the pattern is a *capture*.

In 17.6.3 [temp.variadic] paragraph 6:

A **parameter** pack whose name appears within the pattern of a pack expansion is expanded by that pack expansion. An appearance of the name of a **parameter** pack is only expanded by the innermost enclosing pack expansion. The pattern of a pack expansion shall name one or more **parameter** packs that are not expanded by a nested pack expansion; such **parameter** packs are called *unexpanded parameter packs* in the pattern. All of the **parameter** packs expanded by a pack expansion shall have the same number of arguments specified. An appearance of a name of a **parameter** pack that is not expanded is ill-formed.

Add a new bullet to 17.6.3 [temp.variadic] paragraph 7:

Such an element, in the context of the instantiation, is interpreted as follows:

- if the pack is a template parameter pack, the element is a template parameter of the corresponding kind (type or non-type) designating the type or value from the template argument; **otherwise;**

- if the pack is a function parameter pack, the element is an *id-expression* designating the function parameter that resulted from the instantiation of the pattern where the pack is declared: [: otherwise.](#)
- [if the pack is an *init-capture* pack, the element is an *id-expression* designating the variable declared by the *init-capture* that resulted from the instantiation of the pattern where the pack is declared.](#)

Example

This would simplify all the code that currently relies on `std::tuple` just to solve this problem, in a way that we are already used to seeing pack expansion:

C++17 today	This proposal
<pre>template <class... Args> auto delay_invoke_foo(Args... args) { return [tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) { return std::apply([&](auto const&... args) -> decltype(auto) { return foo(args...); }, tup); }; }</pre>	<pre>template <class... Args> auto delay_invoke_foo(Args... args) { return [...args=std::move(args)]() -> decltype(auto) { return foo(args...); }; }</pre>

Revision History

Changes since r1. Following Core and Evolution guidance, the ellipses for an *init-capture* pack have been moved from following the *init-capture* to preceding it. This is consistent with the existing practice of `...` preceding the name that it introduces.

Changes since r0. Wording changes.

Acknowledgements

Thanks to T.C. for suggesting this solution and pointing out the history of the *init-capture* restriction.

Thanks to Richard Smith, John Spicer, and Daveed Vandevoorde for considering the viability of this change. Thanks to Hubert Tong and Jens Maurer for help with wording.

References

[1] [N3610](#): "Generic lambda-capture initializers, supporting capture-by-move"

[2] [N3648](#): "Wording Changes for Generalized Lambda-capture"

[3] [A problem with generalized captures and pack expansion](#)

[4] [CWG 1760](#): Access of member corresponding to *init-capture*